

Reviewer Pack — Schur-Polynomial Positivity Defect Lower-Bounds Tree-Frege L...

Entry 388ed224a992 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 17:55:55 UTC

Schur-Polynomial Positivity Defect Lower-Bounds Tree-Frege Lines for PHP

Entry ID: 388ed224a992 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 17:55:55 UTC

1. Conjecture

Field A (mathematical branch): Symmetric function theory: Schur-positivity of LR-style coefficients in the expansion of monomial symmetric functions associated to clause-occupancy vectors **Field B** (complexity object): Tree-like Frege (equivalently tree-resolution) refutation line count $L_{TF}(F)$ on the pigeonhole CNF PHP_n^{n+1}

Statement:

For each n in $[3, 9]$, let $F = PHP_n^{n+1}$ and let $v(F)$ in $\mathbb{Z}^n$ be the clause-occupancy vector v_i = number of clauses of F in which variable $x_{\{i\}}$ appears (summed over holes), and form the monomial symmetric function $m_{\{\lambda(v)\}}$ where $\lambda(v)$ is the partition obtained by sorting v . Define the Schur-positivity defect $S(F) = \sum \text{over partitions } \mu \text{ of } n \text{ with } \mu \text{ different from } \lambda(v) \text{ of } \max(0, -\langle m_{\{\lambda(v)\}}, s_{\mu} \rangle)$ where $\langle, \rangle$ denotes the inverse-Kostka pairing computed by Gaussian elimination on the Kostka matrix $K_{\{\mu, \lambda(v)\}}$. Conjecture: $\log_2 L_{TF}(PHP_n^{n+1}) \geq c (S(F) + 1)$ for an absolute constant $c \geq 1/4$, with equality up to a $(1+o(1))$ factor as n grows; one PHP_n^{n+1} with $\log_2 L_{TF} < (S(F)+1)/4$ falsifies.

Rationale (proposer's reasoning):

Schur-positivity defects are sensitive to imbalance in symmetric clause-occupancy structure, and the PHP family's tree-resolution hardness is known to be driven by a symmetric matching obstruction; bridging Kostka inverses (a representation-theoretic invariant rarely seen in proof complexity) with PHP line counts may expose the symmetric kernel of EF lower bounds without invoking natural-proofs-style pseudorandom predicates. The Kostka matrix is integer, lower-triangular under dominance order, and invertible exactly by back-substitution, so the defect is a fully constructive number-theoretic statistic of the CNF, not a categorical invariant.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b4693431df0d119e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across n in $\{3,4,5,6,7,8,9\}$ (7 PHP instances, deterministic so 1 seed each), measure $\log_2(L_TF(PHP_n^{\{n+1\}}))$ and $S(F)$. Conjecture is SUPPORTED iff every instance satisfies $\log_2(L_TF) \geq (S(F)+1)/4$; FALSIFIED if any single n yields $\log_2(L_TF) < (S(F)+1)/4$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Schur positivity pigeonhole principle tree-like Frege lower bounds - Kostka numbers monomial symmetric function resolution refutation PHP - symmetric functions proof complexity tree-resolution clause occupancy

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0304041v1>] $P \neq NP$, propositional proof complexity, and resolution lower bounds for the weak pigeonhole principle - [<http://arxiv.org/abs/2511.20023v1>] Lower Bounds for Bit Pigeonhole Principles in Bounded-Depth Resolution over Parities - [<http://arxiv.org/abs/1611.08382v1>] Conditionally positive definite kernels in Hilbert C^* -modules - [<http://arxiv.org/abs/1501.05996v1>] Double Kostka polynomials and Hall bimodule - [<http://arxiv.org/abs/2404.04038v1>] Refutability as Recursive as Provability - [<http://arxiv.org/abs/1805.05255v1>] Irreducible Characters for the Symmetric Groups and Kostka Matrices - [<http://arxiv.org/abs/2301.02177v3>] The Kromatic Symmetric Function: A K -theoretic Analogue of X_G - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import combinations, permutations

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
```

```

    return A

def inverse_kostka(K, n):
    K_inv = [[0] * n for _ in range(n)]
    for i in range(n):
        K_inv[i][i] = 1 / K[i][i]
        for j in range(i + 1, n):
            factor = K[j][i] / K[i][i]
            for k in range(n):
                K_inv[j][k] -= factor * K_inv[i][k]
    return K_inv

def schur_defect(v, n):
    lambda_v = sorted(v)
    partitions = []
    def generate_partitions(k, current_partition):
        if len(current_partition) == n:
            if sum(current_partition) == k:
                partitions.append(tuple(sorted(current_partition)))
            return
        for i in range(current_partition[-1] if current_partition else 0, k + 1):
            generate_partitions(k - i, current_partition + [i])
    generate_partitions(sum(v), [])

    K = [[0] * len(partitions) for _ in range(len(partitions))]
    for i, mu in enumerate(partitions):
        for j, lambda_v_i in enumerate(lambda_v):
            count = sum(1 for k in range(n) if partitions[i][k] >= lambda_v_i)
            K[i][j] += 1

    K_inv = inverse_kostka(K, len(partitions))
    defect = sum(max(0, -K_inv[i][j]) for i in range(len(partitions)) for j in range(len(lambda_v)))
    return defect

def lex_dp11(F):
    n = len(F)
    stack = []
    assignment = [None] * n
    def backtrack():
        if all(assignment[i] is not None for i in range(n)):
            return True
        var = next(i for i in range(n) if assignment[i] is None)
        for val in (0, 1):
            assignment[var] = val
            stack.append((var, val))
            if all(F[i][j] <= assignment[j] for j in range(n)):
                if backtrack():
                    return True
            stack.pop()
            assignment[var] = None
        return False
    return backtrack()

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [3, 4, 5, 6, 7, 8, 9]

```

```

results = []

for n in n_values:
    F = [[random.randint(0, 1) for _ in range(n)] for _ in range(n + 1)]
    v = [sum(F[i][j] for i in range(n + 1)) for j in range(n)]
    lambda_v = sorted(v)

    partitions = []
    def generate_partitions(k, current_partition):
        if len(current_partition) == n:
            if sum(current_partition) == k:
                partitions.append(tuple(sorted(current_partition)))
            return
        for i in range(current_partition[-1] if current_partition else 0, k + 1):
            generate_partitions(k - i, current_partition + [i])
    generate_partitions(sum(v), [])

    K = [[0] * len(partitions) for _ in range(len(partitions))]
    for i, mu in enumerate(partitions):
        for j, lambda_v_i in enumerate(lambda_v):
            count = sum(1 for k in range(n) if partitions[i][k] >= lambda_v_i)
            K[i][j] += 1

    K_inv = inverse_kostka(K, len(partitions))
    defect = sum(max(0, -K_inv[i][j]) for i in range(len(partitions)) for j in range(len(lambda_v)))

    L_TF = lex_dp11(F)
    if not L_TF:
        return {
            "metric_name": "log2(L_TF)",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "L_TF is falsified"
        }

    log2_L_TF = math.log2(L_TF)
    results.append((n, log2_L_TF, defect))

mean_log2_L_TF = sum(log2_L_TF for _, log2_L_TF, _ in results) / len(results)
std_log2_L_TF = math.sqrt(sum((log2_L_TF - mean_log2_L_TF) ** 2 for _, log2_L_TF, _ in results) / len(results))
support_fraction = sum(1 for _, log2_L_TF, defect in results if log2_L_TF >= (defect + 1) / 4)

return {
    "metric_name": "log2(L_TF)",
    "metric_value": mean_log2_L_TF,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction == 1.0,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))

```

```

results = [run_trial(seed) for seed in seeds]
mean_log2_L_TF = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
std_log2_L_TF = math.sqrt(sum((result["metric_value"] - mean_log2_L_TF) ** 2 for result in results if result["metric_value"] is not None) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if support_fraction == 1.0:
    print(f"RESULT: SUPPORTED mean={mean_log2_L_TF} std={std_log2_L_TF} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"L_TF is falsified\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2496006.py", line 136, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2496006.py", line 108, in run_trial
    L_TF = lex_dp11(F)
           ^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2496006.py", line 77, in lex_dp11
    return backtrack()
           ^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2496006.py", line 71, in backtrack
    if all(F[i][j] <= assignment[j] for j in range(n)):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2496006.py", line 71, in <genexpr>
    if all(F[i][j] <= assignment[j] for j in range(n)):
       ^

```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('i' not defined) inside the lex_dp11 backtrack function before any data was produced, so the pre-registered support condition could not be evaluated. | next: Fix the scoping bug in backtrack() (capture clause index i in the generator) and rerun the deterministic sweep over n in {3,...,9}.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	20396
2	preregistration	claude_max	opus	0	0	6494
3	novelty	claude_max	opus	0	0	3710
4	novelty	claude_max	opus	0	0	8844
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17164
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17448
7	judge	claude_max	opus	0	0	3537

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 77594 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/388ed224a992.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/388ed224a992.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/388ed224a992.tar.gz` (if generated)